

API TEST REPORT

PharmaTrack — Customer-Facing E-Commerce

This test execution summary reports the results of API testing for PharmaTrack's customer-facing e-commerce platform. It delivers the outcomes of automated endpoint validation, data integration workflows, and system responses using Postman, excluding backend admin and pharmacist features.

Document Information

Document Title	API Test Report-Pharmatrack
Version	1.0
Date	5/6/2026
Prepared By	Mohammad Waarits Harahap

1. Executive Summary

PharmaTrack is a modern pharmacy management and digital drug catalog application. To demonstrate professional API test automation capabilities for a QA Engineer Intern role, a comprehensive test suite was developed and executed using the Postman platform.

The testing focused on the customer-facing interface of the PharmaTrack platform, covering core functionalities such as product discovery, category filtering, order creation/checkout validation, and the AI Health Assistant.

The test execution achieved a **100%** success rate, with all 16 requests and 32 assertions passing successfully. The system demonstrated robust functional validation, correctly responding with 201 Created for valid orders and 400 Bad Request with detailed validation messages for malformed inputs (e.g., invalid phone numbers, empty names, or empty carts). The API endpoints responded with an average latency of 1,861 ms (ranging from a minimum of 260 ms for validation checks up to 4.6 seconds for complex AI chat queries).

2. Testing Scope & Boundaries

To ensure clean boundaries and a focused testing strategy, the test suite is strictly categorized into in-scope and out-of-scope areas of the PharmaTrack API.

2.1 In-Scope

The automated tests validate the public, customer-facing interface. The following endpoints and workflows were verified:

- **Product Catalog & Categories:** Retrieving the list of categories, listing active products, filtering products by category slug, searching for products via queries, and verifying correct handling of empty search results.
- **Order & Checkout System:** Submitting online orders (cash payment method), validating input fields (asserting rules on minimum name length, phone number length, and non-empty carts), retrieving order history by customer phone number, and fetching single order details using either the internal UUID or the public order number.
- **AI Health Assistant:** Verifying successful AI chat completions for OTC drug queries, ensuring medical emergency guardrails trigger warnings for serious symptoms, and validating bad request responses for invalid payloads.

2.2 Out-of-Scope

The following features and verification methods are excluded from this test report:

- **Admin & Pharmacist Management:** Internal dashboard APIs for stock adjustment, batch tracking (FEFO expiration control), and sales analytics.
- **Authentication:** Customer registration and login API endpoints, which are handled separately via E2E testing.
- **Verification Type:** The scope is strictly limited to backend API contract validation (status codes, JSON schemas, headers, and response text) and does not cover visual elements, performance load testing, or user-interface (UI) workflows.

3. Test Environment & Tools

The test automation project is built using professional API testing tools and dynamic state management to ensure isolation and realistic end-to-end user journey validation.

3.1 Testing Tools & Setup

- **Postman Client:** Used as the primary platform for constructing requests, organizing test suites, scripting automated assertions, and executing collection runs.
- **Environment Isolation:** A dedicated Postman Environment is utilized to configure base variables, allowing the same test collection to easily target different development, staging, or production environments without modifications to the individual requests.

3.2 Dynamic State & Chaining

To simulate real-world user flows, the collection uses dynamic variable sharing where subsequent requests rely on outputs from previous requests:

- **Product Identification:** The GET Products request dynamically extracts the ID of an active, in-stock product from the response payload and saves it to the environment as `product_id`.
- **Order Chaining:** The POST Create Order request uses the stored `product_id` to submit a checkout transaction. Upon successful creation, the test scripts automatically parse the response to extract the unique order UUID (`LATEST_ORDER_ID`) and the human-readable order number (`LATEST_ORDER_NUMBER`), saving them back to the environment.

- **Order Verification:** The GET Retrieve by Order UUID and GET Retrieve by Order Number requests reference the newly created order details dynamically to verify successful database entry and correct data retrieval.

4. Test Execution Summary

The automated API test collection was executed in a single run. The following table summarizes the endpoints tested, overall functionality and performance results:

Endpoint	Method	Status Code
/api/categories	GET	200 OK
/api/products	GET	200 OK
/api/orders	POST	201 Created
/api/orders/:id	GET	200 OK
/api/orders/{{latest_order_id}}	GET	200 OK
/api/orders/{{latest_order_number}}	GET	200 OK
/api/orders/invalid-uuid-111-222	GET	200 OK
/api/chat	POST	200 OK

Metric	Value
Total Test Scenarios (Requests)	16
Total Assertions Checked	32
Passed Assertions	32
Failed Assertions	0

Overall Pass Rate	100%
Average Response Time	1,861 ms
Minimum Response Time	260 ms
Maximum Response Time	4,600 ms

4.1 Performance Analysis

- **Low-Latency Operations:** Endpoints that do not require external integrations or database write transactions (such as category listings and validation checks) consistently returned within 260 ms - 380 ms.
- **Transactional Operations:** Database write operations (checkout and order creation) returned within 1.5 - 2.8 seconds due to database transactional write latencies.
- **AI-Powered Operations:** The conversational assistant queries showed a higher response time of 2.8 - 4.6 seconds, which is standard for LLM text generation latency.

5. Detailed Test Scenarios & Assertion Results

The test suite consists of 16 distinct scenarios across 3 main modules. All assertions verify HTTP status codes, response structures, and correct error payloads.

5.1 Product Catalog & Categories Module

- **1. GET Categories (Positive):**

Endpoint: /api/categories

Target Status: 200 OK.

Assertions: Verification of the success state (success: true) and validation of the categories array.

```
1  pm.test("Status code is 200", function () {
2    pm.response.to.have.status(200);
3  });
4  pm.test("Response contains categories array", function () {
5    const jsonData = pm.response.json();
6    pm.expect(jsonData.success).to.be.true;
7    pm.expect(jsonData.categories).to.be.an("array");
8    if (jsonData.categories.length > 0) {
9      pm.expect(jsonData.categories[0]).to.have.property("id");
10     pm.expect(jsonData.categories[0]).to.have.property("name");
11     pm.expect(jsonData.categories[0]).to.have.property("slug");
12   }
13 });
```

- **2. GET Products - List Active Products (Positive):**

Endpoint: /api/products?status=active

Target Status: 200 OK.

Assertions: Verify paginated data schema and extract an active product_id for order testing.

```

1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response body matches schema", function () {
5   const jsonData = pm.response.json();
6   pm.expect(jsonData.data).to.be.an("array");
7   pm.expect(jsonData.pagination).to.have.all.keys("total", "page",
8     "limit", "totalPages");
9   if (jsonData.data.length > 0) {
10    const activeProduct = jsonData.data.find(p => p.totalStock > 0);
11    if (activeProduct) {
12      pm.environment.set("product_id", activeProduct.id);
13    }
14  }
15 });

```

- **3. GET Products - Filter Category (Positive):**

Endpoint: /api/products?category=obat-keras

Target Status: 200 OK.

Assertions: Verify that all returned products match the requested category slug (obat-keras).

```

1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("All returned items are in filtered category", function () {
5   const jsonData = pm.response.json();
6   jsonData.data.forEach(product => {
7     pm.expect(product.category.slug).to.equal("obat-keras");
8   });
9 });

```

- **4. GET Products - Search Term Match (Positive):**

Endpoint: /api/products?q=paramex

Target Status: 200 OK.

Assertions: Verify that the search results array is returned based on a query parameter.

```

1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response has search results", function () {
5   const jsonData = pm.response.json();
6   pm.expect(jsonData.data).to.be.an("array");
7 });

```

- **5. GET Products - Search Non-existent (Negative/Boundary):**

Endpoint: /api/products?q=produktidaktersedia

Target Status: 200 OK.

Assertions: Verify that the response returns status 200 with an empty data array.

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response is empty", function () {
5   const jsonData = pm.response.json();
6   pm.expect(jsonData.data).to.be.an("array").that.is.empty;
7 });
```

5.2 Order & Checkout Module

- **6. POST Create Order Success (Positive):**

Endpoint: /api/orders

Target Status: 201 Created.

Assertions: Verify order creation confirmation, order number generation, and save order identifiers.

```
1 pm.test("Status code is 201", function () {
2   console.log("POST Create Order Success RESPONSE:", pm.response.text());
3   pm.response.to.have.status(201);
4 });
5 pm.test("Order confirmation details returned", function () {
6   const jsonData = pm.response.json();
7   pm.expect(jsonData.success).to.be.true;
8   pm.expect(jsonData.orderNumber).to.be.a("string");
9   pm.expect(jsonData.orderId).to.be.a("string");
10  pm.environment.set("LATEST_ORDER_ID", jsonData.orderId);
11  pm.environment.set("LATEST_ORDER_NUMBER", jsonData.orderNumber);
12 });
```

- **7. POST Create Order - Empty Name (Negative):**

Endpoint: /api/orders

Target Status: 400 Bad Request.

Assertions: Verify error response code and exact validation payload error message ('Nama minimal 2 karakter').

```

1 pm.test("Status code is 400", function () {
2     console.log("POST Create Order - Empty Name RESPONSE:", pm.response.
      text());
3     pm.response.to.have.status(400);
4 });
5 pm.test("Error payload explains invalid name input", function () {
6     const jsonData = pm.response.json();
7     pm.expect(jsonData.error).to.equal("Invalid input");
8     pm.expect(jsonData.details.customerName._errors).to.contain("Nama
      minimal 2 karakter");
9 });

```

- **8. POST Create Order - Invalid Phone (Negative):**

Endpoint: /api/orders

Target Status: 400 Bad Request.

Assertions: Verify error response code and validation error message ('Nomor telepon tidak valid').

```

1 pm.test("Status code is 400", function () {
2     pm.response.to.have.status(400);
3 });
4 pm.test("Error payload explains invalid phone input", function () {
5     const jsonData = pm.response.json();
6     pm.expect(jsonData.error).to.equal("Invalid input");
7     pm.expect(jsonData.details.customerPhone._errors).to.contain("Nomor
      telepon tidak valid");
8 });

```

- **9. POST Create Order - Empty Items (Negative):**

Endpoint: /api/orders

Target Status: 400 Bad Request.

Assertions: Verify error response code and validation error message ('Keranjang kosong').

```

1 pm.test("Status code is 400", function () {
2     pm.response.to.have.status(400);
3 });
4 pm.test("Error payload explains empty items", function () {
5     const jsonData = pm.response.json();
6     pm.expect(jsonData.error).to.equal("Invalid input");
7     pm.expect(jsonData.details.items._errors).to.contain("Keranjang
      kosong");
8 });

```

- **10. GET List Orders by Customer Phone (Positive):**

Endpoint: /api/orders?phone=081234567890

Target Status: 200 OK.

Assertions: Verify order history results array matches the customer's phone number.

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Verify orders array matches search criteria", function () {
5   const jsonData = pm.response.json();
6   pm.expect(jsonData.orders).to.be.an("array");
7   jsonData.orders.forEach(order => {
8     pm.expect(order.customerPhone).to.equal("081234567890");
9   });
10  });
```

- **11. GET Retrieve by Order UUID (Positive):**

Endpoint: /api/orders/{{LATEST_ORDER_ID}}

Target Status: 200 OK.

Assertions: Verify correct retrieval of order items and matching UUID.

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Order fields matches detail requirements", function () {
5   const order = pm.response.json().order;
6   pm.expect(order).to.have.property("orderNumber");
7   pm.expect(order.items).to.be.an("array").that.is.not.empty;
8 });
```

- **12. GET Retrieve by Order Number (Positive):**

Endpoint: /api/orders/{{LATEST_ORDER_NUMBER}}

Target Status: 200 OK.

Assertions: Verify correct retrieval structure using the public order number.

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Order matches number", function () {
5   const order = pm.response.json().order;
6   pm.expect(order.orderNumber).to.equal(pm.environment.get
7     ("LATEST_ORDER_NUMBER"));
7 });
```

- **13. GET Retrieve Non-existent UUID (Negative):**

Endpoint: /api/orders/invalid-uuid-111-222

Target Status: 404 Not Found.

Assertions: Verify exact error payload response ('Pesanan tidak ditemukan').

```
1 pm.test("Status code is 404", function () {
2   pm.response.to.have.status(404);
3 });
4 pm.test("Error message is returned", function () {
5   const jsonData = pm.response.json();
6   pm.expect(jsonData.error).to.equal("Pesanan tidak ditemukan");
7 });
```

5.3 AI Health Assistant Module

- **14. POST Chat Successful Query (Positive):**

Endpoint: /api/chat

Target Status: 200 OK.

Assertions: Verify that the AI response text is populated for general OTC medicine questions.

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("AI response text is populated", function () {
5   const jsonData = pm.response.json();
6   pm.expect(jsonData.text).to.be.a("string").that.is.not.empty;
7 });
```

- **15. POST Chat Prompt for Serious Symptom (Positive/Guardrails):**

Endpoint: /api/chat

Target Status: 200 OK.

Assertions: Verify that the AI output contains critical emergency instructions (advising the user to call emergency services or visit the nearest IGD).

```
1 pm.test("Status code is 200", function () {
2   pm.response.to.have.status(200);
3 });
4 pm.test("Response mentions emergency instruction", function () {
5   const text = pm.response.json().text.toLowerCase();
6   const emergencyTriggered = text.includes("igd") || text.includes
7     ("dokter") || text.includes("rumah sakit") || text.includes
8     ("hubungi 119");
9   pm.expect(emergencyTriggered).to.be.true;
10 });
```

- **16. POST Chat Invalid Payload (Negative):**

Endpoint: /api/chat

Target Status: 400 Bad Request.

Assertions: Verify error response code and standard error message ('Invalid messages array').

```
1 pm.test("Status code is 400", function () {
2   console.log("POST Chat Invalid Payload RESPONSE:", pm.response.text
3     ());
4   pm.response.to.have.status(400);
5 });
6 pm.test("Error message matches invalid messages array", function () {
7   const jsonData = pm.response.json();
8   pm.expect(jsonData.error).to.equal("Invalid messages array");
9 });
```

6. Conclusion

Based on the execution results of the 16 test scenarios, the customer-facing interface of the PharmaTrack API demonstrates high functional reliability and correct behavior under validation constraint checks.

6.1 Conclusions

- **Functional Correctness:** The API correctly handles all happy path scenarios and rejects invalid inputs with precise validation error payloads, confirming that backend database schema constraints are properly enforced.
- **Transactional Consistency:** Database transactions (order creation and retrieval) successfully update and retrieve records without data integrity issues.
- **AI Latency Optimization:** Conversational queries to the AI Health Assistant take up to 4.6 seconds to complete. It is recommended to implement client-side streaming or caching mechanisms for common queries to improve the user experience and reduce perceived waiting times.